

Kiro.dev

Understand your existing codebase

Before starting new development, Kiro analyzes your existing code and generates three foundational documents:

- **structure.md** (codebase architecture),
- **tech.md** (technical stack and patterns),
- **product.md** (business context and requirements).

This gives you and your team a clear baseline understanding that informs all subsequent specification work. Existing codebases can now take advantage of this new paradigm.

Analyze and plan your project

When you provide a project prompt in spec mode, Kiro's AI doesn't immediately start coding. Instead, it performs deep analysis to understand your requirements, identifies potential challenges, and creates comprehensive planning documents.

Generate comprehensive planning documentation

From a simple prompt, Kiro creates detailed specification files including:

- **Requirements Analysis** - Breaking down your prompt into specific, actionable requirements
- **Technical Design** - Architecture decisions, technology choices, and implementation approach
- **Task Breakdown** - Granular development tasks with clear acceptance criteria

Collaborate with AI effectively

Kiro saves specification files in your project directory as readable markdown files. You can review, edit, and refine them before any code is written. This creates natural checkpoints for collaboration with team members or stakeholders.

Maximize coding context and efficiency

When it's finally time to write code, Kiro references these specification files rather than cluttering your context window with exploratory conversation. This means maximum context space is available for the actual coding task.

From a single prompt, you can collaborate with Kiro on creating concise specifications that include clear requirements, system architecture, tech stack considerations, and implementation approach

While Kiro incorporates many advanced features to assist with development, being an Agentic IDE, the manual code review processes are not listed as one of its core concepts. Below are the **core concepts** for your review:

- **Autonomous and supervised code generation and modification**
- **Specs for breaking down complex features into manageable implementation plans**
- **Hooks that automate repetitive tasks based on IDE events**
- **Steering capabilities for fine-tuning AI behavior to match team standards**
- **MCP (Model Context Protocol) for extending functionality with specialized tools**
- **Comprehensive codebase understanding and contextual assistance**

Key Components

1. Requirements definition

Kiro uses EARS (Easy Approach to Requirements Syntax) notation for capturing requirements. This structured format provides clear, testable criteria for development, specifying conditions and expected behaviors

2. Design considerations
3. Implementation tasks
4. References to supporting documentation

Specs bridge the gap between conceptual product requirements and technical implementation details, ensuring alignment and reducing development iterations. Kiro generates three key files that form the foundation of each specification:

Kiro's approach in creating specs

requirements.md - captures user stories and acceptance criteria in structured notation.

design.md - documents technical architecture, sequence diagrams, and implementation considerations.

tasks.md - provides a detailed implementation plan with discrete, and trackable tasks.

Specs act as living documents that evolve with your project, maintaining synchronization between documentation and code. This approach ensures that teams can collaborate effectively while preserving institutional knowledge and design decisions.

Workflow Overview

The Kiro workflow follows a logical progression through four distinct phases, each building upon the previous to create a comprehensive development process. At each stage, Kiro's AI capabilities assist in generating and refining specifications while maintaining focus on project goals.

Workflow Phases

The workflow follows a logical progression with decision points between phases, ensuring each step is properly completed before moving to the next.

The workflow remains highly adaptable, allowing for iterative refinement at any stage, **while Kiro handles the complexity of automatically adjusting any dependencies** when modifications are made, making the entire development process both flexible and efficiently managed.

Requirements Phase

Requirements in Kiro are captured using the Easy Approach to Requirements Syntax (EARS) notation, providing clear, testable criteria for development. Each requirement follows a structured format that specifies conditions and expected behaviors, making it easier to translate requirements into actual features.

EARS notation provides a structured format for writing clear, testable requirements. In a spec's **requirements.md** file, each requirement follows this pattern:

WHEN [condition/event]

THE SYSTEM SHALL [expected behavior]

The following example demonstrates this pattern:

Condition or event

WHEN

a user submits a form with invalid data

Expected behavior

THE SYSTEM SHALL

display validation errors next to the relevant fields

Clarity

Requirements are unambiguous and easy to understand .

Testability

Each requirement can be directly translated into test cases

Traceability

Individual requirements can be tracked through implementation

Completeness

The format encourages thinking through all conditions and behaviors

Kiro helps you transform vague feature requests into these well-structured requirements, making the development process more efficient and reducing misunderstandings between product and engineering teams.

Here is a sample of **requirement.md** created by Kiro based on the specs of Soccer Voting Application:

Soccer Voting Application Specification # file: REQUIREMENT.MD

User Stories and Requirements

User Authentication

WHEN a user attempts to register

THE SYSTEM SHALL validate:

Age verification (18+ years)

Valid identification

Unique voter registration code

Previous voting history

WHEN a user logs in

THE SYSTEM SHALL verify credentials and voting eligibility

Voting System

WHEN an authorized user accesses the voting interface

THE SYSTEM SHALL:

Display current candidates with images

Show real-time voting statistics

Enable single-vote submission

Prevent duplicate voting

Vote Management

WHEN an administrator creates a voting event

THE SYSTEM SHALL:

Allow configuration of voting duration

Support multiple candidate entries

Enable verification code generation

Provide result tallying options

Results Display

WHEN any user views results

THE SYSTEM SHALL:

Show real-time vote counts

Display percentage distributions

present historical voting data

Design Phase

The design phase in Kiro focuses on translating requirements into technical specifications and architecture decisions. Through the `design.md` file, teams can document system components, interactions, and implementation considerations in a format that's both human-readable and AI-processable.

Kiro's specs offer a structured approach to design documentation, making it easier to understand and collaborate on complex systems. The **`design.md`** file is where you document technical architecture, sequence diagrams, and implementation considerations. It's a great place to capture the big picture of how the system will work, including the components and their interactions.

Here is a sample of **`design.md`** created by Kiro based on the specs of Soccer Voting Application:

Technical Design Specification, file: DESIGN.MD

Architecture

- Frontend: React.js with real-time updates
- Backend: Node.js with WebSocket support
- Database: MongoDB for user/vote storage
- Authentication: JWT with refresh tokens

Components

1. Authentication Service

- User registration
- Login management
- Session handling
- Verification code validation

2. Voting Engine

- Vote submission handler
- Real-time vote counter
- Duplicate vote prevention
- Results aggregator

3. Admin Dashboard

- Voting event creation
- Candidate management
- Code generation system
- Results export

Security Measures

- End-to-end encryption for votes
- Rate limiting on API endpoints
- Input validation and sanitization
- Audit logging for all actions

Implementation Phase

The implementation phase in Kiro breaks down high-level requirements into discrete, manageable tasks. The **tasks.md** file serves as a dynamic roadmap, tracking progress and maintaining alignment between specifications and actual development work.

The **tasks.md** file is where you provide a detailed implementation plan with discrete, trackable tasks and sub-tasks. Each task is clearly defined, with a clear description, expected outcome, and any necessary resources or dependencies. Kiro's specs offer a structured approach to implementation plans, making it easier to understand and collaborate on complex systems.

Kiro provides a task execution interface for **tasks.md** files that displays real-time status updates. Tasks are updated as in-progress or completed, allowing you to efficiently track implementation progress and maintain an up-to-date view of your development status.

Here is a sample of **tasks.md** created by Kiro based on the specs of Soccer Voting Application:

Implementation Tasks, file: TASKS.MD

Phase 1: Authentication System

- [] Implement user registration flow
- [] Create verification code system
- [] Build login mechanism
- [] Develop session management

Phase 2: Core Voting System

- [] Create candidate management interface
- [] Build vote submission system
- [] Implement real-time vote counting
- [] Develop duplicate vote prevention

Phase 3: Admin Features

- [] Build admin dashboard
- [] Create voting event management
- [] Implement verification code generator
- [] Develop results export system

Phase 4: Results Display

- [] Create real-time results view
- [] Implement historical data display
- [] Build statistics dashboard
- [] Add automated updates

Technical Stack Details, file: TECH-STACK.MD

Frontend

- React.js 18+
- Socket.io-client
- Material-UI components
- Chart.js for statistics

Backend

- Node.js with Express
- Socket.io for real-time
- MongoDB with Mongoose
- Redis for caching

Infrastructure

- AWS ECS for container hosting
- CloudFront for content delivery
- DynamoDB for session management
- S3 for image storage

DEMO VIDEO TRANSCRIPT: Soccer-Voting-App

Hi, everyone. Today I'll be demonstrating Kiro's spec generation feature, and I'll talk a bit about how it can streamline your development process. Specs generation feature is a powerful tool designed to help you build and document complex features in a structured way. It provides a formalized approach to the design and implementation process, allowing you to iterate with Kiro on requirements, design, and implementation tasks.

Let me show you how this works with a real example.

I will **demonstrate by adding a search feature** to the **soccer voting app** that I've been working on. It is an **XJS soccer boarding app** that I've built where **users can browse through clubs from Europe's top 5 European leagues**, namely **Premier League, La Liga, Bundesliga, Serie A, and Liga**, and **vote for their favorites**. The problem is that we have over **50 teams across these leagues**, and as you can see, **I have to endlessly scroll to find a specific team like Barcelona, Bayern Munich, or Paris Saint Germain**. And now I need to add the search functionality to it.

But as a developer, **there are several important decisions I need to make prior to starting with it. Should I filter on the client side or the server side? How do I handle the UI? Where should we place the UI in our existing layout? How do we maintain performance with potentially hundreds of teams, adding on to the application?** Etc. Instead of diving straight into code, **let me show you how Kiro helps think through this systematically**. I'm going to pivot to my existing workspace that I've opened, and I'm **going to bring up the chat UX using command option B to toggle the secondary sidebar**. And, I'm going to simply **start off by writing**. A simple problem saying **PROMPT: "create a spec that adds a search feature. That filters Team names. Team by names. As users type"**. So, now, once I start executing this command, the **agent initially looks at the product or tech and the structure documentation**, which are part of the agent **steering rules**. It will then suggest I'll start creating a spec for me. (**mkdir -p .kiro/specs/team-search-filter**) ←--- requirements.md created for the existing code base.

ARNING: Retry #2, retrying within 10.0s..

> Created the spec at /workspaces/sample-aiml-solution-labs/workshop/soccer-voting-app/TEAM-SEARCH-FEATURE-SPEC.MD

The specification includes:

- ****User story**** - Clear description of who needs it and why
- ****Functional requirements**** - Search input behavior, filtering logic, and scope
- ****Technical requirements**** - Component structure and filtering implementation
- ****Acceptance criteria**** - Testable scenarios for validation

- **Implementation notes** - State management and performance considerations
- **Testing requirements** - Unit and integration test cases
- **Effort estimate** - 3-5 hours total

The feature filters teams by name in real-time as users type, with case-insensitive substring matching and a clear button to reset the search.

► Credits: 0.44 • Time: 1m 22s

Watch how Kiro analyzes a code base and generates a comprehensive specification. It's not just generic advice, it's tailored to my specific next JS app structure and existing components. *It starts off by creating a new directory within the Kiro folder, which consists of a **specs folder** now, followed by the **Spec name** which is **Team Search printer**. I'm going to accept this command. So now *if I pivot into the Kiro directory, I should see a spec folder with the **TeamSearch** filter as the spec name*. Now, once this is created, *what Kiro will do is it will now go ahead and analyze my codebase to create the **requirements file for me***. Secure, as you say, as you see here, it's suggesting I'm reading the files, it's going through and navigating through different components *that I have **API routes that I've defined***, and *now it's starting to create the requirements do for me*. So here there's a **requirements.md** file which gets created. So right now, it's working on creating it. Let's just give a moment for it to complete. And, there we go. *Now you see that the requirements file has been populated with certain information*. It includes *detailed requirements written in a testable format*. So it includes requirement stories, user stories, and even acceptance criteria. **PROMPT: "As a user, I want to search for teams by name as a type, so that I can quickly find my team."** That's the exact problem that we're trying to solve. And here's the acceptance criteria for the task. Now, *once that is done, Kiro actually waits for user input to understand whether the user wants something else*. So *it says, does the requirements look good? If so, we can move on to the design*. So, I'm going to select. Skip question because it looks good to me.*

Now, the *next step in the generation process is to create the design file*. Now you can pivot to the same specs directory, and we'll shortly observe that it now starts generating the design file. So before it does that, *it says, I'd like to get a feedback on the requirements that I've created. Are there any changes or additions?* Looks good.

PROMPT:

[help] 19% > Please proceed with the desing of team-search

.: Thinking...

WARNING: Retry #2, retrying within 10.0s..

I'll create the following file: `/workspaces/sample-aiml-solution-labs/workshop/soccer-voting-app/TEAM-SEARCH-DESIGN.MD` (using tool: write)

Purpose: Create detailed design document for team search feature implementation

```
+ 1: # Team Search Feature - Design Document
+ 2:
+ 3: ## Component Architecture
+ 4:
+ 5: ### Component Hierarchy
+ 6: ```
+ 7: VotingPage
+ 8: |— TeamSearchBar (new)
+ 9: |— TeamList
+ 10: |— TeamCard[]
+ 11: ```
+ 12:
+ 13: ## Component Design
+ 14:
+ 15: ### TeamSearchBar Component
+ 16:
+ 17: ```javascript
+ 18: // src/components/TeamSearchBar.jsx
+ 19: import { TextField, InputAdornment, IconButton } from '@mui/material';
+ 20: import { Search, Clear } from '@mui/icons-material';
+ 21:
+ 22: export default function TeamSearchBar({ searchTerm, onSearchChange, onClear }) {
+ 23:   return (
+ 24:     <TextField
+ 25:       fullWidth
```

```

+ 26:   placeholder="Search teams..."
+ 27:   value={searchTerm}
+ 28:   onChange={(e) => onSearchChange(e.target.value)}
+ 29:   InputProps={{
+ 30:     startAdornment: (
+ 31:       <InputAdornment position="start">
+ 32:         <Search />
+ 33:       </InputAdornment>
+ 34:     ),
+ 35:     endAdornment: searchTerm && (
+ 36:       <InputAdornment position="end">
+ 37:         <IconButton onClick={onClear} size="small" aria-label="Clear search">
+ 38:           <Clear />
+ 39:         </IconButton>
+ 40:       </InputAdornment>
+ 41:     ),
+ 42:   }}
+ 43:   sx={{ mb: 3 }}
+ 44:   aria-label="Search teams"
+ 45: />
+ 46: );
+ 47: }
+ 48: ```
+ 49:
+ 50: ### VotingPage Component (Updated)
+ 51:
+ 52: ```javascript
+ 53: // src/pages/VotingPage.jsx
+ 54: import { useState, useMemo } from 'react';

```

```
+ 55: import TeamSearchBar from '../components/TeamSearchBar';
+ 56: import TeamList from '../components/TeamList';
+ 57:
+ 58: export default function VotingPage() {
+ 59:   const [teams, setTeams] = useState([]);
+ 60:   const [searchTerm, setSearchTerm] = useState("");
+ 61:
+ 62:   const filteredTeams = useMemo(() => {
+ 63:     if (!searchTerm) return teams;
+ 64:     return teams.filter(team =>
+ 65:       team.name.toLowerCase().includes(searchTerm.toLowerCase())
+ 66:     );
+ 67:   }, [teams, searchTerm]);
+ 68:
+ 69:   const handleClearSearch = () => setSearchTerm("");
+ 70:
+ 71:   return (
+ 72:     <div>
+ 73:       <TeamSearchBar
+ 74:         searchTerm={searchTerm}
+ 75:         onSearchChange={setSearchTerm}
+ 76:         onClear={handleClearSearch}
+ 77:       />
+ 78:       {filteredTeams.length === 0 ? (
+ 79:         <p>No teams found matching "{searchTerm}"</p>
+ 80:       ) : (
+ 81:         <TeamList teams={filteredTeams} />
+ 82:       )}
+ 83:     </div>
```

```

+ 84: );
+ 85: }
+ 86: ```
+ 87:
+ 88: ## Visual Design
+ 89:
+ 90: ### Layout
+ 91: ```
+ 92: |_____|
+ 93: | 🔍 Search teams...    ×  | ← Search Bar
+ 94: |_____|
+ 95: |_____|
+ 96: | Manchester United    [Vote] 45% | ← Team Card
+ 97: |-----|
+ 98: | Manchester City      [Vote] 32% |
+ 99: |-----|
+ 100: | Real Madrid          [Vote] 23% |
+ 101: |_____|
+ 102: ```
+ 103:
+ 104: ### Styling Specifications
+ 105:
+ 106: ```javascript
+ 107: // Theme values (Material-UI)
+ 108: const styles = {
+ 109:   searchBar: {
+ 110:     height: '48px',
+ 111:     marginBottom: '24px',
+ 112:     backgroundColor: '#ffffff',

```

```

+ 113: borderRadius: '8px',
+ 114: '& .MuiOutlinedInput-root': {
+ 115:   '&:hover fieldset': {
+ 116:     borderColor: 'primary.main',
+ 117:   },
+ 118:   '&.Mui-focused fieldset': {
+ 119:     borderWidth: '2px',
+ 120:   },
+ 121: },
+ 122: },
+ 123: emptyState: {
+ 124:   textAlign: 'center',
+ 125:   color: 'text.secondary',
+ 126:   padding: '48px 16px',
+ 127:   fontSize: '16px',
+ 128: },
+ 129: };
+ 130: ```
+ 131:
+ 132: ## State Management
+ 133:
+ 134: ### State Structure
+ 135: ```javascript
+ 136: {
+ 137:   searchTerm: string,    // Current search input
+ 138:   teams: Team[],        // All teams from API
+ 139:   filteredTeams: Team[]  // Computed from teams + searchTerm
+ 140: }
+ 141: ```

```

```
+ 142:
+ 143: ### Data Flow
+ 144: ```
+ 145: User Input → searchTerm state → useMemo filter → filteredTeams → TeamList render
+ 146: ```
+ 147:
+ 148: ## API Integration
+ 149:
+ 150: ### No Backend Changes Required
+ 151: - Filtering happens client-side
+ 152: - Existing team fetch endpoint unchanged
+ 153: - Search state is local to frontend
+ 154:
+ 155: ## Accessibility
+ 156:
+ 157: ### ARIA Attributes
+ 158: ```javascript
+ 159: <TextField
+ 160:   aria-label="Search teams"
+ 161:   inputProps={{
+ 162:     'aria-describedby': 'search-help-text'
+ 163:   }}
+ 164: />
+ 165:
+ 166: <IconButton
+ 167:   aria-label="Clear search"
+ 168:   onClick={onClear}
+ 169: />
+ 170:
```



```
+ 171: <div role="status" aria-live="polite">
+ 172: {filteredTeams.length} teams found
+ 173: </div>
+ 174: `` `
+ 175:
+ 176: ### Keyboard Navigation
+ 177: - Tab: Focus search input
+ 178: - Escape: Clear search (optional enhancement)
+ 179: - Enter: No action (prevents form submission)
+ 180:
+ 181: ## Performance Optimization
+ 182:
+ 183: ### useMemo Hook
+ 184: `` ` javascript
+ 185: const filteredTeams = useMemo(() => {
+ 186:   if (!searchTerm) return teams;
+ 187:   return teams.filter(team =>
+ 188:     team.name.toLowerCase().includes(searchTerm.toLowerCase())
+ 189:   );
+ 190: }, [teams, searchTerm]);
+ 191: `` `
+ 192:
+ 193: **Why useMemo?**
+ 194: - Prevents re-filtering on every render
+ 195: - Only recomputes when teams or searchTerm changes
+ 196: - Critical for maintaining real-time vote updates
+ 197:
+ 198: ## Edge Cases
+ 199:
```

- + 200: ### Empty States
- + 201: 1. ****No search term****: Show all teams
- + 202: 2. ****No matches****: Show "No teams found matching '[term]'"
- + 203: 3. ****No teams loaded****: Show loading spinner (existing behavior)
- + 204:
- + 205: ### Special Characters
- + 206: - Handle special regex characters in search (e.g., "(", ")")
- + 207: - Use simple string includes (no regex needed)
- + 208:
- + 209: ### Long Team Names
- + 210: - Team cards handle text overflow with ellipsis
- + 211: - Search matches full name, not truncated display
- + 212:
- + 213: ## Testing Strategy
- + 214:
- + 215: ### Unit Tests
- + 216: `` ` ` javascript
- + 217: describe('TeamSearchBar', () => {
- + 218: test('renders search input', () => {});
- + 219: test('calls onSearchChange when typing', () => {});
- + 220: test('shows clear button when text present', () => {});
- + 221: test('calls onClear when clear button clicked', () => {});
- + 222: });
- + 223:
- + 224: describe('VotingPage filtering', () => {
- + 225: test('filters teams by name substring', () => {});
- + 226: test('filter is case-insensitive', () => {});
- + 227: test('shows all teams when search is empty', () => {});
- + 228: test('shows empty state when no matches', () => {});

```
+ 229: });
+ 230: ```
+ 231:
+ 232: ### Integration Tests
+ 233: ```javascript
+ 234: test('user can search and vote for filtered team', async () => {
+ 235:   render(<VotingPage />);
+ 236:
+ 237:   const searchInput = screen.getByPlaceholderText('Search teams...');
+ 238:   fireEvent.change(searchInput, { target: { value: 'Manchester' } });
+ 239:
+ 240:   expect(screen.getByText('Manchester United')).toBeInTheDocument();
+ 241:   expect(screen.queryByText('Real Madrid')).not.toBeInTheDocument();
+ 242:
+ 243:   const voteButton = screen.getByRole('button', { name: /vote/i });
+ 244:   fireEvent.click(voteButton);
+ 245:
+ 246:   // Assert vote was submitted
+ 247: });
+ 248: ```
+ 249:
+ 250: ## Implementation Checklist
+ 251:
+ 252: - [ ] Create TeamSearchBar component
+ 253: - [ ] Add searchTerm state to VotingPage
+ 254: - [ ] Implement filtering logic with useMemo
+ 255: - [ ] Add empty state message
+ 256: - [ ] Style search bar with Material-UI theme
+ 257: - [ ] Add ARIA labels for accessibility
```

- + 258: - [] Write unit tests for TeamSearchBar
- + 259: - [] Write integration tests for filtering
- + 260: - [] Test with real-time vote updates
- + 261: - [] Test keyboard navigation
- + 262: - [] Update IMPL-TASKS.MD with completion
- + 263:
- + 264: ## File Structure
- + 265:
- + 266: ```
- + 267: src/
- + 268: |— components/
- + 269: | |— TeamSearchBar.jsx (new)
- + 270: | |— TeamSearchBar.test.jsx (new)
- + 271: | |— TeamList.jsx (existing)
- + 272: | |— TeamCard.jsx (existing)
- + 273: |— pages/
- + 274: | |— VotingPage.jsx (modified)
- + 275: | |— VotingPage.test.jsx (modified)
- + 276: |— utils/
- + 277: |— filterTeams.js (optional helper)
- + 278: ```
- + 279:
- + 280: ## Dependencies
- + 281:
- + 282: ### Existing (from TECH-STACK.MD)
- + 283: - React 18+
- + 284: - Material-UI components
- + 285: - Socket.io-client (for real-time updates)

- + 286:
- + 287: ### New Dependencies
- + 288: None - uses existing Material-UI components
- + 289:
- + 290: ## Deployment Notes
- + 291:
- + 292: - No database migrations required
- + 293: - No API changes required
- + 294: - Frontend-only change
- + 295: - Can be deployed independently
- + 296: - No feature flag needed (low risk)

Creating: /workspaces/sample-aiml-solution-labs/workshop/soccer-voting-app/TEAM-SEARCH-DESIGN.MD

- Completed in 0.1s

❏ Thinking...

WARNING: Retry #2, retrying within 10.0s..

> Created the design document at /workspaces/sample-aiml-solution-labs/workshop/soccer-voting-app/TEAM-SEARCH-DESIGN.MD

The design includes:

Component Architecture

- TeamSearchBar component with Material-UI TextField
- Updated VotingPage with search state and filtering logic

Visual Design

- Layout mockup showing search bar above team list

- Styling specifications using Material-UI theme

Implementation Details

- Complete React component code
- useMemo optimization for filtering
- ARIA attributes for accessibility

Testing Strategy

- Unit tests for search component
- Integration tests for filtering + voting

Edge Cases & Performance

- Empty states handling
- Client-side filtering (no backend changes)
- Keyboard navigation support

Ready to implement with all code examples provided.

► Credits: 0.44 • Time: 54s

So it will now go ahead and start building the design file for me, as you see here. So, *the design file is now being created*. Now, *the very, a very handful way to interact with specs in general is also through **the navbar created in these files***. So initially here, you'd see **Step 1 (one) marked as requirements**. *Step 2 is once you've created the requirements, you can either regenerate it based on any manual edits that you've done or any manual user stories that you'd like to include*. **The second step will be to create the design**. So right now, the agent is working **on creating the design file for me**. So let's wait and give it a moment. To create the design file. *And there you go. **design.md** now consists of the **architecture, client-side filtering, component-based design** based on how I've already structured my application, **State management** includes a **responsive design**. Here's **the search input component** that the *application should use, and so on*. **The technical design** shows how the **league filter** will work alongside the **search filter**.*

And, now **the final step** that is required **is implementation tasks**. *What tasks need to be executed to achieve this outcome*. So here again, it will come back and **ask the user for input**, suggesting, **does the design look good?** If so, *we can move on to* **the implementation phase**. I'm going to say

yes, move on to the next step. And while it does so, it now starts creating the task file. So here, if you see in the spec directory, there is the task file that is now being created. Now, while this file is being created, I'll also show you another way to access the spec files. **So if you navigate to the activity bar, you can select the Kiro icon** from there and **select the specs folder from the Kiro tab.** And from here, you could also select **the plan**, which will take you to **the requirements file, the design, and the task file.** So I'm going to **pivot to the task file** now. So here, as you see, it has now **created A few tasks for me** and it consists of **subtasks** as well. So first is **creating the search input component.** The 2nd The task includes implementing **team filtering logic.** And **the team filtering logic has subtasks within it for creating a custom hook for team filtering,** and no results state handling, **integrate search functionality,** and so on. So, now what I'm going to do is, it again, asks for user input in case you need to modify anything during this process, you could do so by chat by chatting with the agent. So I'm **going to say, yes, the tasks look good.** OK, awesome.

[help] 21% > The technical design shows how the league filter will work alongside the search filter. now the final step that is required is implementation tasks. please move to the implementation phase, which tasks need to be implemented.

.: Thinking...

WARNING: Retry #2, retrying within 10.0s..

.: Thinking...

Using auto

I'll create the following file: /workspaces/sample-aiml-solution-labs/workshop/soccer-voting-app/TEAM-SEARCH-IMPLEMENTATION-TASKS.MD (using tool: write)

Purpose: Create detailed implementation task breakdown for team search and league filter features

- + 1: # Team Search & League Filter - Implementation Tasks
- + 2:
- + 3: ## Phase 1: Core Search Functionality
- + 4:
- + 5: ### Task 1.1: Create TeamSearchBar Component
- + 6: ****File**:** `src/components/TeamSearchBar.jsx`
- + 7: - [] Create functional component with TextField
- + 8: - [] Add Search icon (start adornment)
- + 9: - [] Add Clear button (end adornment, conditional)

- + 10: - [] Implement onChange handler
- + 11: - [] Implement onClear handler
- + 12: - [] Add ARIA labels
- + 13: - [] Apply Material-UI styling
- + 14:
- + 15: **Estimated Time**: 1 hour
- + 16:
- + 17: **Task 1.2: Add Search State to VotingPage**
- + 18: **File**: `src/pages/VotingPage.jsx`
- + 19: - [] Add `searchTerm` state with useState
- + 20: - [] Create `handleSearchChange` function
- + 21: - [] Create `handleClearSearch` function
- + 22: - [] Import TeamSearchBar component
- + 23: - [] Add TeamSearchBar to JSX above TeamList
- + 24:
- + 25: **Estimated Time**: 30 minutes
- + 26:
- + 27: **Task 1.3: Implement Team Filtering Logic**
- + 28: **File**: `src/pages/VotingPage.jsx`
- + 29: - [] Create `filteredTeams` with useMemo
- + 30: - [] Implement case-insensitive substring match
- + 31: - [] Pass filteredTeams to TeamList component
- + 32: - [] Add empty state message for no results
- + 33:
- + 34: **Estimated Time**: 45 minutes
- + 35:
- + 36: ---
- + 37:
- + 38: **Phase 2: League Filter Functionality**

- + 39:
- + 40: ### Task 2.1: Create LeagueFilter Component
- + 41: **File**: `src/components/LeagueFilter.jsx`
- + 42: - [] Create functional component with Select dropdown
- + 43: - [] Add "All Leagues" default option
- + 44: - [] Map league options from teams data
- + 45: - [] Implement onChange handler
- + 46: - [] Add label "Filter by League"
- + 47: - [] Apply Material-UI styling
- + 48:
- + 49: **Estimated Time**: 1 hour
- + 50:
- + 51: ### Task 2.2: Add League Filter State
- + 52: **File**: `src/pages/VotingPage.jsx`
- + 53: - [] Add `selectedLeague` state with useState (default: 'all')
- + 54: - [] Create `handleLeagueChange` function
- + 55: - [] Import LeagueFilter component
- + 56: - [] Add LeagueFilter to JSX
- + 57:
- + 58: **Estimated Time**: 30 minutes
- + 59:
- + 60: ### Task 2.3: Combine Search and League Filters
- + 61: **File**: `src/pages/VotingPage.jsx`
- + 62: - [] Update useMemo to filter by both search AND league
- + 63: - [] Apply league filter first, then search filter
- + 64: - [] Update empty state message to reflect both filters
- + 65: - [] Ensure filters work independently and together
- + 66:
- + 67: **Estimated Time**: 45 minutes

- + 68:
- + 69: ---
- + 70:
- + 71: ## Phase 3: UI/UX Enhancements
- + 72:
- + 73: ### Task 3.1: Create Filter Container Layout
- + 74: **File**: `src/pages/VotingPage.jsx`
- + 75: - [] Create Grid/Box container for filters
- + 76: - [] Position search bar (left, 70% width)
- + 77: - [] Position league filter (right, 30% width)
- + 78: - [] Add responsive breakpoints for mobile
- + 79: - [] Add spacing between filters
- + 80:
- + 81: **Estimated Time**: 45 minutes
- + 82:
- + 83: ### Task 3.2: Add Results Count Display
- + 84: **File**: `src/pages/VotingPage.jsx`
- + 85: - [] Add Typography component showing count
- + 86: - [] Display "Showing X of Y teams"
- + 87: - [] Add aria-live="polite" for screen readers
- + 88: - [] Position below filters
- + 89:
- + 90: **Estimated Time**: 30 minutes
- + 91:
- + 92: ### Task 3.3: Style Empty States
- + 93: **File**: `src/pages/VotingPage.jsx`
- + 94: - [] Create EmptyState component (optional)
- + 95: - [] Add icon for empty state
- + 96: - [] Style message text (centered, gray)

- + 97: - [] Add "Clear filters" button when both active
- + 98:
- + 99: ****Estimated Time****: 45 minutes
- + 100:
- + 101: ---
- + 102:
- + 103: **## Phase 4: Testing**
- + 104:
- + 105: **### Task 4.1: Unit Tests - TeamSearchBar**
- + 106: ****File****: `src/components/TeamSearchBar.test.jsx`
- + 107: - [] Test component renders
- + 108: - [] Test onChange callback fires
- + 109: - [] Test clear button appears with text
- + 110: - [] Test clear button calls onClear
- + 111: - [] Test ARIA labels present
- + 112:
- + 113: ****Estimated Time****: 1 hour
- + 114:
- + 115: **### Task 4.2: Unit Tests - LeagueFilter**
- + 116: ****File****: `src/components/LeagueFilter.test.jsx`
- + 117: - [] Test component renders
- + 118: - [] Test options populate correctly
- + 119: - [] Test onChange callback fires
- + 120: - [] Test "All Leagues" default selected
- + 121:
- + 122: ****Estimated Time****: 45 minutes
- + 123:
- + 124: **### Task 4.3: Integration Tests - Combined Filtering**
- + 125: ****File****: `src/pages/VotingPage.test.jsx`

- + 126: - [] Test search filters teams by name
- + 127: - [] Test league filter filters teams by league
- + 128: - [] Test both filters work together (AND logic)
- + 129: - [] Test empty state shows when no matches
- + 130: - [] Test clear search resets search only
- + 131: - [] Test changing league resets correctly
- + 132: - [] Test filtered teams can be voted on
- + 133:
- + 134: ****Estimated Time****: 2 hours
- + 135:
- + 136: **### Task 4.4: Integration Tests - Real-time Updates**
- + 137: ****File****: `src/pages/VotingPage.test.jsx`
- + 138: - [] Test vote counts update with active search
- + 139: - [] Test vote counts update with active league filter
- + 140: - [] Test new teams appear in filtered view
- + 141: - [] Test WebSocket updates don't reset filters
- + 142:
- + 143: ****Estimated Time****: 1 hour
- + 144:
- + 145: ---
- + 146:
- + 147: **## Phase 5: Accessibility & Polish**
- + 148:
- + 149: **### Task 5.1: Keyboard Navigation**
- + 150: ****File****: `src/pages/VotingPage.jsx`
- + 151: - [] Test Tab navigation through filters
- + 152: - [] Add Escape key to clear search (optional)
- + 153: - [] Ensure focus management works correctly
- + 154: - [] Test with keyboard-only navigation

- + 155:
- + 156: ****Estimated Time****: 45 minutes
- + 157:
- + 158: **### Task 5.2: Screen Reader Testing**
- + 159: - ☐ Test with NVDA/JAWS
- + 160: - ☐ Verify filter changes announced
- + 161: - ☐ Verify results count announced
- + 162: - ☐ Verify empty state announced
- + 163:
- + 164: ****Estimated Time****: 1 hour
- + 165:
- + 166: **### Task 5.3: Mobile Responsiveness**
- + 167: ****File****: `src/pages/VotingPage.jsx`
- + 168: - ☐ Test filters stack vertically on mobile
- + 169: - ☐ Test touch interactions work
- + 170: - ☐ Adjust spacing for small screens
- + 171: - ☐ Test with Chrome DevTools mobile view
- + 172:
- + 173: ****Estimated Time****: 45 minutes
- + 174:
- + 175: ---
- + 176:
- + 177: **## Phase 6: Documentation & Deployment**
- + 178:
- + 179: **### Task 6.1: Update Documentation**
- + 180: ****Files****: `IMPL-TASKS.MD`, `README.md`
- + 181: - ☐ Mark search feature tasks complete
- + 182: - ☐ Add league filter to feature list
- + 183: - ☐ Document filter behavior

- + 184: - ☐ Add screenshots/GIFs of filters
- + 185:
- + 186: ****Estimated Time****: 30 minutes
- + 187:
- + 188: **### Task 6.2: Code Review Preparation**
- + 189: - ☐ Run linter and fix issues
- + 190: - ☐ Ensure all tests pass
- + 191: - ☐ Check code coverage (aim for 80%+)
- + 192: - ☐ Write PR description
- + 193:
- + 194: ****Estimated Time****: 30 minutes
- + 195:
- + 196: **### Task 6.3: Deployment**
- + 197: - ☐ Merge to main branch
- + 198: - ☐ Deploy to staging environment
- + 199: - ☐ Smoke test on staging
- + 200: - ☐ Deploy to production
- + 201: - ☐ Monitor for errors
- + 202:
- + 203: ****Estimated Time****: 1 hour
- + 204:
- + 205: ---
- + 206:
- + 207: **## Summary**
- + 208:
- + 209: **### Total Estimated Time: 16-18 hours**
- + 210:
- + 211: **### Task Breakdown by Phase:**
- + 212: - Phase 1 (Search): 2.25 hours

- + 213: - Phase 2 (League Filter): 2.25 hours
- + 214: - Phase 3 (UI/UX): 2 hours
- + 215: - Phase 4 (Testing): 4.75 hours
- + 216: - Phase 5 (Accessibility): 2.5 hours
- + 217: - Phase 6 (Documentation): 2 hours
- + 218:
- + 219: ### Priority Order:
- + 220: 1. **P0 (Must Have)**: Phase 1, Phase 2, Task 4.3
- + 221: 2. **P1 (Should Have)**: Phase 3, Phase 4 (remaining), Phase 5.1
- + 222: 3. **P2 (Nice to Have)**: Phase 5.2, Phase 5.3, Phase 3.3
- + 223:
- + 224: ### Dependencies:
- + 225: - Task 2.3 depends on Task 1.3 and Task 2.2
- + 226: - Task 3.1 depends on Task 1.1 and Task 2.1
- + 227: - Phase 4 depends on Phase 1 and Phase 2 completion
- + 228: - Phase 6 depends on all previous phases
- + 229:
- + 230: ### Team Assignment Suggestion:
- + 231: - **Developer 1**: Phase 1 + Phase 2 (core functionality)
- + 232: - **Developer 2**: Phase 3 + Phase 5 (UI/UX + accessibility)
- + 233: - **QA Engineer**: Phase 4 (testing)
- + 234: - **Tech Lead**: Phase 6 (review + deployment)
- + 235:
- + 236: ### Risk Mitigation:
- + 237: - Start with Phase 1 only, validate with stakeholders
- + 238: - Add Phase 2 after Phase 1 approval
- + 239: - Phase 3 enhancements can be done in parallel
- + 240: - Testing can start once Phase 1 is complete

**Creating: /workspaces/sample-aiml-solution-labs/workshop/soccer-voting-app/TEAM-
SEARCH-IMPLEMENTATION-TASKS.MD**

- Completed in 0.1s

The implementation plan includes:

6 Phases with 18 Tasks:

1. Phase 1: Core Search (2.25 hrs)

- TeamSearchBar component
- Search state management
- Filtering logic

2. Phase 2: League Filter (2.25 hrs)

- LeagueFilter component
- League state management
- Combined filter logic

3. Phase 3: UI/UX (2 hrs)

- Filter layout
- Results count
- Empty states

4. Phase 4: Testing (4.75 hrs)

- Unit tests for both components
- Integration tests for combined filtering
- Real-time update tests

5. Phase 5: Accessibility (2.5 hrs)

- Keyboard navigation

- Screen reader support
- Mobile responsiveness

6. Phase 6: Documentation (2 hrs)

- Update docs
- Code review
- Deployment

Total: 16-18 hours with clear priorities (P0/P1/P2) and team assignment suggestions.

► Credits: 0.52 • Time: 1m 31s

So, *it has now completed creating the spec file of **the structure** for me.* And it says, ***here's the requirements document which defines the user stories, acceptance criteria,*** and so on. The next file as we saw was **the design file** which *created a comprehensive design.* **That outlines the architecture components** you are designed to be used, **testing strategy**, and so on. Finally, there was **the task or the implementation plan**, *which was created with a detailed task list with specific coding tasks uh organized in a logical sequence.*

Team Search & League Filter - Implementation Tasks file: **TEAM-SEARCH-IMPLEMENTATION-TASKS.MD**

Phase 1: Core Search Functionality

Task 1.1: Create TeamSearchBar Component

****File**:** `src/components/TeamSearchBar.jsx`

- [] Create functional component with TextField
- [] Add Search icon (start adornment)
- [] Add Clear button (end adornment, conditional)
- [] Implement onChange handler
- [] Implement onClear handler
- [] Add ARIA labels
- [] Apply Material-UI styling

****Estimated Time****: 1 hour

Task 1.2: Add Search State to VotingPage

****File****: `src/pages/VotingPage.jsx`

- [] Add `searchTerm` state with `useState`
- [] Create `handleSearchChange` function
- [] Create `handleClearSearch` function
- [] Import `TeamSearchBar` component
- [] Add `TeamSearchBar` to JSX above `TeamList`

****Estimated Time****: 30 minutes

Task 1.3: Implement Team Filtering Logic

****File****: `src/pages/VotingPage.jsx`

- [] Create `filteredTeams` with `useMemo`
- [] Implement case-insensitive substring match
- [] Pass `filteredTeams` to `TeamList` component
- [] Add empty state message for no results

****Estimated Time****: 45 minutes

Phase 2: League Filter Functionality

Task 2.1: Create LeagueFilter Component

****File****: `src/components/LeagueFilter.jsx`

- [] Create functional component with `Select` dropdown

- [] Add "All Leagues" default option
- [] Map league options from teams data
- [] Implement onChange handler
- [] Add label "Filter by League"
- [] Apply Material-UI styling

****Estimated Time**:** 1 hour

Task 2.2: Add League Filter State

- **File**:** `src/pages/VotingPage.jsx`
- [] Add `selectedLeague` state with useState (default: 'all')
 - [] Create `handleLeagueChange` function
 - [] Import LeagueFilter component
 - [] Add LeagueFilter to JSX

****Estimated Time**:** 30 minutes

Task 2.3: Combine Search and League Filters

- **File**:** `src/pages/VotingPage.jsx`
- [] Update useMemo to filter by both search AND league
 - [] Apply league filter first, then search filter
 - [] Update empty state message to reflect both filters
 - [] Ensure filters work independently and together

****Estimated Time**:** 45 minutes

Phase 3: UI/UX Enhancements

Task 3.1: Create Filter Container Layout

****File**:** `src/pages/VotingPage.jsx`

- [] Create Grid/Box container for filters
- [] Position search bar (left, 70% width)
- [] Position league filter (right, 30% width)
- [] Add responsive breakpoints for mobile
- [] Add spacing between filters

****Estimated Time**:** 45 minutes

Task 3.2: Add Results Count Display

****File**:** `src/pages/VotingPage.jsx`

- [] Add Typography component showing count
- [] Display "Showing X of Y teams"
- [] Add aria-live="polite" for screen readers
- [] Position below filters

****Estimated Time**:** 30 minutes

Task 3.3: Style Empty States

****File**:** `src/pages/VotingPage.jsx`

- [] Create EmptyState component (optional)
- [] Add icon for empty state
- [] Style message text (centered, gray)
- [] Add "Clear filters" button when both active

****Estimated Time**:** 45 minutes

Phase 4: Testing

Task 4.1: Unit Tests - TeamSearchBar

****File**:** `src/components/TeamSearchBar.test.jsx`

- [] Test component renders
- [] Test onChange callback fires
- [] Test clear button appears with text
- [] Test clear button calls onClear
- [] Test ARIA labels present

****Estimated Time**:** 1 hour

Task 4.2: Unit Tests - LeagueFilter

****File**:** `src/components/LeagueFilter.test.jsx`

- [] Test component renders
- [] Test options populate correctly
- [] Test onChange callback fires
- [] Test "All Leagues" default selected

****Estimated Time**:** 45 minutes

Task 4.3: Integration Tests - Combined Filtering

****File**:** `src/pages/VotingPage.test.jsx`

- [] Test search filters teams by name
- [] Test league filter filters teams by league
- [] Test both filters work together (AND logic)
- [] Test empty state shows when no matches

- [] Test clear search resets search only
- [] Test changing league resets correctly
- [] Test filtered teams can be voted on

****Estimated Time**:** 2 hours

Task 4.4: Integration Tests - Real-time Updates

****File**:** `src/pages/VotingPage.test.jsx`

- [] Test vote counts update with active search
- [] Test vote counts update with active league filter
- [] Test new teams appear in filtered view
- [] Test WebSocket updates don't reset filters

****Estimated Time**:** 1 hour

Phase 5: Accessibility & Polish

Task 5.1: Keyboard Navigation

****File**:** `src/pages/VotingPage.jsx`

- [] Test Tab navigation through filters
- [] Add Escape key to clear search (optional)
- [] Ensure focus management works correctly
- [] Test with keyboard-only navigation

****Estimated Time**:** 45 minutes

Task 5.2: Screen Reader Testing

- [] Test with NVDA/JAWS

- [] Verify filter changes announced

- [] Verify results count announced

- [] Verify empty state announced

****Estimated Time**:** 1 hour

Task 5.3: Mobile Responsiveness

****File**:** `src/pages/VotingPage.jsx`

- [] Test filters stack vertically on mobile

- [] Test touch interactions work

- [] Adjust spacing for small screens

- [] Test with Chrome DevTools mobile view

****Estimated Time**:** 45 minutes

Phase 6: Documentation & Deployment

Task 6.1: Update Documentation

****Files**:** `IMPL-TASKS.MD`, `README.md`

- [] Mark search feature tasks complete

- [] Add league filter to feature list

- [] Document filter behavior

- [] Add screenshots/GIFs of filters

****Estimated Time**:** 30 minutes

Task 6.2: Code Review Preparation

- [] Run linter and fix issues
- [] Ensure all tests pass
- [] Check code coverage (aim for 80%+)
- [] Write PR description

****Estimated Time**:** 30 minutes

Task 6.3: Deployment

- [] Merge to main branch
- [] Deploy to staging environment
- [] Smoke test on staging
- [] Deploy to production
- [] Monitor for errors

****Estimated Time**:** 1 hour

Summary

Total Estimated Time: 16-18 hours

Task Breakdown by Phase:

- Phase 1 (Search): 2.25 hours
- Phase 2 (League Filter): 2.25 hours
- Phase 3 (UI/UX): 2 hours
- Phase 4 (Testing): 4.75 hours
- Phase 5 (Accessibility): 2.5 hours

- Phase 6 (Documentation): 2 hours

Priority Order:

1. **P0 (Must Have)**: Phase 1, Phase 2, Task 4.3
2. **P1 (Should Have)**: Phase 3, Phase 4 (remaining), Phase 5.1
3. **P2 (Nice to Have)**: Phase 5.2, Phase 5.3, Phase 3.3

Dependencies:

- Task 2.3 depends on Task 1.3 and Task 2.2
- Task 3.1 depends on Task 1.1 and Task 2.1
- Phase 4 depends on Phase 1 and Phase 2 completion
- Phase 6 depends on all previous phases

Team Assignment Suggestion:

- **Developer 1**: Phase 1 + Phase 2 (core functionality)
- **Developer 2**: Phase 3 + Phase 5 (UI/UX + accessibility)
- **QA Engineer**: Phase 4 (testing)
- **Tech Lead**: Phase 6 (review + deployment)

Risk Mitigation:

- Start with Phase 1 only, validate with stakeholders
- Add Phase 2 after Phase 1 approval
- Phase 3 enhancements can be done in parallel
- Testing can start once Phase 1 is complete

REQUIREMENTS DESIGN TASK LIST

One bullet is a task, that is selectable, and clickable

For EXECUTION

Each tied to its own set of requirements. So now comes the fun part, **actually building this feature.** So instead of staring at a blank file wondering where to start, I'll **click on the first task in our implementation plan, which is to create the search input component.** So as soon as I select that, **you'll see that Kiro marks it as in progress,** and there's **another button next to it called view changes.** As Kiro starts incrementally changing the code related to it. So let's take a look at what it's creating. **You could, for now, select new changes, but since it's still reading through the project directory and understanding where exactly to create the component, you'll see the execution file changes to be empty. Let's just give it a moment and I'm going to close this and it's now started to implement the search input component for me.** So let's see. So it's creating **the search input component. Within the components directory.** This file creation should just wrap up in a moment and there we go. **So this is impressive. It is creating the search component that perfectly matches our coding style.** Notice how it's creating **the search input component in the same directory that we've been using under apps and components.** It **also uses the same tailwind CSS class that we've been using throughout the app for consistent styling.** And, **it even handles focus states and any accessibility attributes that are required.** Now, once this is implemented, I'm going to **go back into the tasks,** and here you'll see that **the task is now completed. You can view the changes.** Now, it will show you all the changes that have been made, followed by viewing the execution as well in the chat session. And similarly, **now I'm going to start implementing the second task, which is to implement this team filtering logic now.** So I'm going to **pause the video uh for the execution to complete and then resume.** OK. So now, **Kiro has completed executing tasks 2 and 3, which is for implementing team filtering logic and the search functionality into the voting view.** So here you could see that each of these tasks are **marked as completed,** where when Kiro starts to move on to the **next task, this systematic approach ensures we don't miss anything** and if we're working with a team, everyone can see exactly what's being implemented and what's still trending. Now, I'm going to quickly **navigate to my application** to see this in action. So let me **pivot to this application and I'm going go into the voting view.** When I do this, you'll now see **a search bar right into your voting view.** So I'm going search for Barcelona. When I do that, **it filters exactly when I start typing,** which is one of the main requirements I had, **and it now shows a couple of teams within LaLiga,** which consists of the word Barcelona. So I'm going **select one, and submit the vote for it.** Now, the next thing to test would also be **filtering via league.** So I'm going say, **Bundesliga.** When I do this, **this is still pending.** So now let's refine our spec to also ensure that we are filtering **not just by the team name but also by the leagues.**

So, I'm going pivot back to my application here and I'm going to **Pull up the chat** and I'm going **ask to refine the spec so that the user can filter by Leagues too.** So once I do that, Kiro will not only update one of my files, but **it will actually update all of the files that are associated with it.** So **if a requirement changes, it not only makes updates to it, but also goes ahead and make the respective changes in the design and as file respectively.** So right now, it's looking at the requirements file. Let's give it a moment for that to update. Now, it's **looking through the design file** to see **if we have already implemented something similar or what needs to change.** Followed by **the task file.** The **task file now goes ahead and updates the requirements file.** Let's see, the fact that is being implemented so far. No changes in yet. OK. So I do see **a new requirement that has been added to the file, so I'm going say good, and if I look at requirement file as a user, I want to filter teams by league** so that I can quickly find teams from a specific competition. So once this is done now, it's also **updating the design file.** So this kind of **coherent approach means that you as a**

developer can experiment with different feature ideas and edge cases before writing a single line of code. We can even *share this spec with your team* to *get feedback early in the process as you're iterating through your product*. So, let's see the changes it's been making in the design file now. And there we go. There's another new component that is added to the application, which is the league filter component and a custom hook to manage the combined filtering logic amongst the team name and the league name as well. Now it goes ahead and updates the final file, which here is **the task file**. So let's see that in action in just a moment. OK, there we go. So we now have a couple new tasks, **namely task number 4**, *which implements league filtering functionality*. So I'm going to **go ahead and start implementing this task** and **pause the video until it completes**. Another handy feature here is **while the execution was running**, you would have seen the status **here as task is queued**. *Once an existing execution is completed, the next task in the queue will take precedence and starts implementing*. So right now, that's the change that is occurring. So I'm going to **pause the video briefly while the execution completes**. OK. So Kiro has now wrapped up implementing task number 4, which is **implementing the league filtering functionality**. It *has completed all the 4 subtasks for creating the league filter, enhancing the team filtering hook, and finally integrating it into the voting view*. **Let's take a quick peek at the changes it has made**.

So I'm going **select the view changes option** from here and **just look at how seamlessly it has integrated the league filter component right into your page.tsx file**. Knowing how it's currently **built for my application**, without any additional guidance. It has also updated the league filter in the app components directory, so maintaining folder structure and design pattern, followed by updating the no-result state management and finally, using the use team filter hook. Now, let me put back to the application to see this in **action**. So, **Let me open this up on the full screen**. So here now when I pivot to **the start voting view**, I *now have options to filter via leagues*. **So I'm going quickly search for, let's say Bundesliga here**. It now *shows me all the teams associated with it*. Not only that, I can now search for teams like Bayern, so as I start typing. It shows me *the team name*. *That I've selected and I can vote for it now*.

And there you go. **In just a few minutes**, I *went from a vague idea to a fully implemented feature with clear documented specification, a thoughtful technical design that Kiro created for me, clean, maintainable code following the best practices and guidelines that I had already created for my app and in my project, a polished user experience*. And, what I love about **this feature is that it handles the tedious planning and documentation work while still giving me full control over the implementation**. **It helps me think through edge cases** I might have missed and **make sure I'm building features that integrates well with my existing code base**. That's all I had for today.

Best Practices - Spec-driven Development in Kiro

Successful implementation of Kiro's spec-driven development relies on following established best practices that maximize efficiency and collaboration. These practices ensure that teams can fully leverage Kiro's capabilities while maintaining code quality and project organization.

The following practices have been proven effective across various project sizes and team structures, helping organizations maintain consistency and quality in their development processes.

Managing shared specs within a team

Teams can share artifacts (steering, hooks, and specs) by leveraging Git submodules or package references. Here are some options to consider:

Create a central specs repository

Establish a dedicated repository for shared artifacts that multiple projects can reference

Use Git submodules or package references

Link your central artifacts to individual projects using Git submodules, package references, or symbolic links depending on your development environment

Implement cross-repository workflows

Develop processes for proposing, reviewing, and updating shared artifacts that affect multiple projects

Handling multiple specs

Create separate specs for different features within your project rather than maintaining a single spec for your entire codebase.

For example, in an e-commerce application, organize your specs like this:

This approach helps you to:

- •Work on features independently without conflicts
- •Maintain focused, manageable spec documents
- •Iterate on specific functionality without affecting other areas
- •Collaborate with team members on different features simultaneously

Importing existing requirements

When importing requirements or designs from another system (such as JIRA, Confluence, or Word documents), you have two options:

Using MCP integration

Connect directly through an MCP server that supports STDIO to import requirements into your spec session

Manual import

Copy existing requirements (e.g., foo-prfaq.md) into a new file in your repository, open a spec chat session, and use the command "#foo-prfaq.md Generate a spec from it"

ask execution

Perform task-wise execution

While you can execute all tasks in your **tasks.md** file at once, it is recommended executing tasks individually for better control and results

Managing task execution in a collaborative environment

Option 1: Update tasks via tasks.md

1. Open your tasks.md file
2. Choose "Update tasks"
3. Kiro will automatically mark completed tasks

Option 2: Use Kiro's scanning capability

- In a spec session, ask Kiro: "Check which tasks are already complete"
- Kiro will analyze your codebase and identify implemented functionality
- Tasks will be automatically marked as completed

Creating separate specs for different features within a project is **a recommended approach**, rather than maintaining a single spec for the entire codebase. This approach allows for **independent work on features, maintains focused and manageable spec documents**, and **enables easier collaboration on different features simultaneously**.

<https://kiro.dev/docs/getting-started/>

[Your first project - IDE - Docs - Kiro](#)

(